SIMATS ENGINEERING

Descriptive Written Exam

NAME: Charuthya P
REG NO : 192421148
COURSE NAME: Software Testing **COURSE CODE: CSA3711**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*
Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3
4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3

5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3
11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3

12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3
15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3
18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3

20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3

1. Testing as an Engineering Activity – Banking Application

Testing should be treated as a systematic engineering activity rather than random checking. For a banking application, the testing process should begin with clear requirements, test planning, systematic test design, execution, defect reporting and regression testing.
Systematic test design can be introduced by preparing test scenarios and test cases from requirements. Techniques such as equivalence partitioning, boundary value analysis, decision table testing and state transition testing can replace ad-hoc testing. Positive, negative and edge cases should be included.
Testing tools can be used for test management, automation, defect tracking and regression testing. Standards and defined procedures should specify how test cases are reviewed, executed, documented and reported.
For example, in a money-transfer module, test cases should verify valid transfers, insufficient balance, invalid account number, maximum transfer limit, duplicate transactions and network interruption. Automated regression tests can then be executed whenever changes are made.
This engineering approach improves defect detection because testing becomes repeatable, measurable and traceable. It also improves software quality by reducing missed scenarios and repeated defects.

2. Structured Testing Process for ATM Withdrawal

A structured testing process should be followed throughout the Software Development Life Cycle (SDLC). The major stages are requirement analysis, test planning, test design, test environment preparation, test execution, defect reporting, retesting, regression testing and test closure.
During requirement analysis, the tester identifies withdrawal rules such as card validation, PIN verification, balance checking and withdrawal limits. During test planning, scope, resources, schedule and testing techniques are defined. Test cases are designed before execution.
For ATM withdrawal, test cases should include a valid card and PIN, invalid PIN, insufficient balance, amount above the daily limit, invalid amount, cancelled transaction and successful cash withdrawal. Boundary values should also be tested.
When a failure occurs, the tester records the defect with steps to reproduce, expected result, actual result and severity. After the developer fixes it, the tester performs retesting and then regression testing to ensure that existing functions are not affected.
A consistent process reduces defects because every important activity is planned and documented. Expected outcomes include fewer missed requirements, better defect detection, reliable ATM transactions and improved software quality.

3. Errors, Defects and Failures in a Web Application

Error, defect and failure are related but different testing terms. An error is a human mistake made during requirements, design or coding. A defect is the flaw introduced into the software because of that error. A failure occurs when the software executes the defect and produces an incorrect result.
For example, suppose a developer writes the login condition incorrectly and allows an invalid password. The programmer's incorrect logic is an error, the incorrect code is a defect, and allowing an unauthorized user to log in is a failure.
Similarly, in a transaction system, if the developer misunderstands the transaction limit, the resulting incorrect implementation is a defect. When a user performs a transaction and the system accepts an amount beyond the permitted limit, it becomes a failure.
Improper understanding of these terms can cause unclear defect reports and communication problems between testers and developers. Defect reports should clearly mention the module, steps to reproduce, expected result, actual result, severity, priority and supporting evidence.
Clear terminology improves defect tracking, communication and faster resolution.

4. Defect Clustering and Test Case Prioritization

Defect clustering is the principle that a small number of modules often contain a large proportion of defects. Therefore, testing effort should be concentrated on high-risk and defect-prone areas.
The team should study previous defect data and identify modules with frequent or severe defects. These modules should receive more detailed testing. Test cases can be prioritized based on risk, business importance, defect history, complexity and frequency of use.
For example, in an online banking application, login, fund transfer and payment modules may have more defects because they involve security and complex business rules. These modules should be tested before less critical modules.
Techniques such as risk-based testing, boundary value analysis, equivalence partitioning and regression testing can improve coverage. High-priority test cases should be executed first so that critical defects are detected early.
Applying defect clustering and prioritization makes testing more efficient because limited testing time is focused on areas where failures are more likely or more costly. This improves quality and reduces project delays.

5. Defect Classification and Defect Repository

A defect repository is a centralized collection of information about defects found during testing and after release. It should be actively used to prevent repeated defects.
Defects can originate from incorrect requirements, design mistakes, coding errors, integration problems, configuration issues or environmental conditions. They may be classified as functional, performance, usability, security, interface, data or compatibility defects.
The repository should record defect ID, module, description, severity, priority, steps to reproduce, root cause, status, resolution and release information. Testers can analyze this data to identify repeated patterns.
For example, if the repository shows repeated validation defects in a payment module, additional validation test cases should be created and included in regression testing. Root-cause analysis can also identify why the same type of defect is repeatedly introduced.
Regular review of repository data, root-cause analysis and updating regression suites can prevent recurring defects. This improves defect prevention, test design and overall software quality.

6. Mobile Banking Application – Fund Transfer Crashes

Frequent crashes during fund transfer indicate that systematic testing and root-cause analysis are required. The tester should first collect crash details, logs, device information, input data and steps that reproduce the problem.
Testing should cover valid and invalid account numbers, insufficient balance, incorrect transaction details, network interruption, timeout, duplicate requests, large amounts and simultaneous transactions. Different devices, operating systems and network conditions should also be tested.
For example, if the application crashes when the network disconnects after the user presses Transfer, the tester should reproduce the situation and check whether the application handles the timeout safely. The defect should be documented with expected and actual results.
Unit testing, integration testing, system testing, regression testing and negative testing should be included. Automated regression tests can verify that fixed transfer functions continue to work after future changes. Systematic testing reduces failures by covering normal, abnormal and boundary conditions. Better logging, error handling, code reviews and continuous regression testing can prevent similar crashes in future releases.

7. Testing and Debugging

Testing and debugging are different but complementary activities. Testing is performed to identify failures and defects, while debugging is performed mainly by developers to locate the cause of a defect and correct the code.

A tester executes a test case and observes an unexpected result. The tester reports the defect with clear evidence. The developer investigates the code, identifies the root cause, modifies the program and verifies the correction.

For example, if a payroll system calculates an incorrect salary, testing detects the wrong output. Debugging then examines the calculation logic and identifies the coding mistake responsible for the incorrect value.

Confusing testing with debugging can cause testers to spend time modifying code instead of reporting defects objectively. It can also lead to incomplete testing because developers may assume that fixing a defect means the entire system is correct.

A structured approach is to perform testing, report defects, assign them to developers, debug and fix them, perform retesting, and finally conduct regression testing. This separation improves defect detection and software quality.

8. Cross-Browser Testing Strategy

Cross-browser testing verifies that a web application behaves correctly across different browsers, browser versions, operating systems and screen sizes. Inconsistent behavior may occur because browsers interpret HTML, CSS, JavaScript and APIs differently.

The testing strategy should first identify the browsers and devices most commonly used by customers. Test cases should then cover important functions such as login, navigation, forms, payments and responsive layouts.

For example, an e-commerce website may work correctly in one browser but the checkout button may not respond in another because of a JavaScript or CSS compatibility issue. The same test scenario should be executed across the selected browser combinations.

Tools such as Selenium and browser-based automation platforms can support repeated cross-browser testing. Manual testing is also useful for visual and usability checks. Test matrices should record browser, version, operating system and test result.

Systematic cross-browser testing improves reliability by detecting compatibility defects before release and ensuring a consistent user experience.

9. Defect Classification in Payroll Salary Calculation

Incorrect salary calculations caused by logical errors are functional defects, specifically calculation or business-logic defects. They occur when the implemented calculation rules do not correctly represent the required salary rules.
For example, if the system should calculate salary as basic pay plus allowances minus deductions but incorrectly adds a deduction, the result will be wrong. The defect may have originated from a requirement misunderstanding, design mistake or coding error.
Testers should classify the defect according to its type, severity and priority. Test cases should cover normal salary values, minimum and maximum values, different allowance combinations, deductions, overtime and boundary conditions.
Equivalence partitioning can divide salary inputs into valid categories, while boundary value analysis can test limits. Decision tables are useful when salary depends on several conditions.
Preventive measures include requirement reviews, code reviews, unit testing, automated calculation tests and regression testing. Proper defect classification helps testers select suitable test techniques and prevents similar defects in future releases.

10. Structured Test Case Design Instead of Random Testing

Random testing alone cannot provide systematic coverage. Testing should be redesigned as an engineering activity in which test cases are derived from requirements, risks and expected user behavior.
A good test case should contain a test case ID, objective, preconditions, test data, steps, expected result and actual result. Test cases should include positive, negative, boundary and error-handling scenarios.
For example, for a login module, tests should include valid username and password, invalid password, empty fields, locked account, incorrect username and password combination, and maximum input length.
Techniques such as equivalence partitioning, boundary value analysis, decision table testing and state transition testing can improve test effectiveness. Risk-based prioritization should ensure that important functions are tested first.
Structured test design improves defect detection because it reduces missed scenarios and makes testing repeatable. It also supports regression testing and provides clear evidence of test coverage.

11. Data Inconsistency in a Healthcare Application

Data inconsistency occurs when different modules of a healthcare application display or store conflicting information. Such defects may originate from incorrect requirements, database mapping errors, synchronization problems, interface defects or incorrect update logic.

Systematic testing should verify that the same patient information remains consistent across registration, appointment, billing, laboratory and medical-record modules. Integration testing is especially important because information moves between modules.

For example, if a patient's name or medical record number is updated in the registration module but the old value appears in another module, the tester should compare the data flow and identify the point where the inconsistency occurs.

Defects should be tracked with clear descriptions, affected modules, test data, expected and actual results, severity and evidence. Regression testing should be performed after corrections.

Data validation, database testing, integration testing and automated consistency checks can improve data integrity and reduce repeated defects.

12. Defect Repository for Repeated Release Defects

A defect repository can help analyze why defects continue to appear in different releases. The repository should contain complete and accurate historical information about defects.

Testers should examine defect categories, affected modules, root causes, severity, release, frequency and resolution. Patterns can then be identified. For example, if similar input-validation defects appear in several releases, it indicates a weakness in requirements, coding standards or regression testing.

Repository information should be used to create new test cases and update existing regression suites. High-frequency and high-severity defect areas should receive additional testing.

Root-cause analysis should be performed for recurring defects. Code reviews, requirement reviews, reusable validation components and automation can prevent the same mistakes.

Therefore, a defect repository should not be treated only as a record-keeping system. It should actively support test planning, defect prevention and continuous quality improvement.

13. Web-Based Learning System – Peak Usage Failure

Failure during peak usage indicates that important performance scenarios were not adequately tested. Testing should include realistic user loads and conditions before deployment.
The test plan should identify expected concurrent users, response-time requirements, critical functions and peak usage periods. Performance testing should include load testing, stress testing and endurance testing.
For example, if thousands of students access an online examination or learning portal at the same time, the tester should verify login, course access, video loading, assignment submission and database response under realistic load.
Test coverage should include both normal usage and peak conditions. Monitoring CPU, memory, database performance, network utilization and response time helps identify bottlenecks.
Better test planning and performance coverage improve reliability by exposing scalability problems before release. Capacity planning, performance baselines and regular performance testing should be included in future releases.

14. Early Tester Involvement in SDLC

Testers should be involved from the early stages of the SDLC rather than only after coding is completed. Early involvement helps identify defects when they are cheaper and easier to correct.
During requirements analysis, testers can review requirements for ambiguity, missing conditions and testability. During design, they can identify integration and usability risks. During development, testers can prepare test cases and support unit and integration testing.
For example, in an online payment application, a tester involved during requirements analysis may identify a missing requirement for transaction timeout handling. Correcting the requirement before coding prevents a later defect.
Early involvement also improves collaboration between developers, testers, business analysts and other team members. Reviews, shared documentation, continuous communication and early test planning should be adopted.
This approach reduces rework, improves requirements quality and increases overall defect prevention.

15. Incomplete Test Coverage

A system may produce correct results for some cases and fail for others when test coverage is incomplete. This means important inputs, conditions, paths or requirements have not been adequately tested.
Test coverage should be improved by mapping test cases to requirements and identifying untested conditions. Positive, negative, boundary, exception and integration scenarios should be included.
For example, a banking transfer may work for normal amounts but fail when the amount equals the daily limit, when the balance is insufficient or when the network disconnects during the transaction. These are important conditions that should be tested.
Equivalence partitioning, boundary value analysis, decision tables and state transition testing can improve test design. Risk-based testing can prioritize critical functionality.
Proper coverage increases reliability because more realistic and abnormal conditions are validated before release. Coverage reports and regular review of missing scenarios can further improve testing.

16. Improving Defect Reporting in a Logistics Application

Poor defect reporting delays identification and resolution because developers may not understand the problem clearly. A defect report should provide accurate and reproducible information.
Each report should contain defect ID, title, module, environment, preconditions, steps to reproduce, test data, expected result, actual result, severity, priority and supporting evidence such as screenshots or logs.
For example, instead of reporting “tracking is not working,” a clear report would state the shipment ID, the exact steps followed, the expected tracking status and the actual status displayed.
Defects should be classified consistently and assigned appropriate severity and priority. Standard templates and defect-management tools can improve communication.
Clear reporting reduces misunderstandings, helps developers reproduce issues quickly and supports accurate tracking. It also makes historical defect information useful for future test planning.

17. Process-Based Testing for Consistency

Process-based testing uses standardized activities, procedures and documentation so that testing is performed consistently across different modules.
The team should define common test planning, test case design, review, execution, defect reporting, retesting and regression procedures. Standard templates and checklists should be used.
For example, if the login, payment and order modules all use the same test case format and defect-reporting process, testers can easily understand results and compare quality across modules.
Test standards, review checklists, common entry and exit criteria, reusable test cases and centralized defect tracking can maintain consistency. Regular reviews and team training should also be conducted.
Standardized processes improve quality by reducing variations in testing, preventing missed activities and making results more reliable. Process-based testing is especially useful when multiple testers work on different modules.

18. Boundary Testing and Untested Edge Cases

Edge cases are conditions near or outside normal operating limits. They are often missed when testing focuses only on typical inputs. Boundary value analysis is useful for identifying such defects.
For example, if an application accepts an age from 18 to 60, tests should include 17, 18, 19, 59, 60 and 61.
Similar boundary tests should be applied to transaction amounts, file sizes, dates and input lengths.
A system that works for normal values may fail at minimum or maximum limits because of incorrect conditions or data handling. Test cases should therefore explicitly include lower boundary, upper boundary, just-inside and just-outside values.
Boundary testing should be combined with equivalence partitioning and negative testing. Edge cases should also be included in regression suites after defects are found.
This approach increases test coverage and reduces the possibility of failures appearing after deployment.

19. Defect Documentation and Repository Management

Proper defect documentation is essential for tracking, analyzing and resolving software issues. A defect repository provides a centralized place to store information about defects throughout the software life cycle.
Each defect should contain an identification number, title, module, description, environment, steps to reproduce, expected result, actual result, severity, priority, status, root cause and resolution.
For example, if a payment defect is reported without reproduction steps, the developer may not be able to reproduce it. A structured report provides sufficient information for quick investigation.
Repository data can be analyzed to identify defect-prone modules, recurring problems, common root causes and release trends. This information can then be used to improve test cases and preventive measures.
Standard defect templates, regular repository reviews and consistent classification improve documentation quality. A well-maintained repository supports communication, traceability and continuous improvement.

20. Redesigning Testing for Banking Transaction Processing

Repeated transaction failures indicate that the existing testing process is not sufficiently systematic. A structured testing approach should cover requirements, test design, execution, defect management and regression testing.
The first step is to identify transaction requirements such as valid transactions, account validation, balance checking, transaction limits, duplicate prevention and failure recovery. Test cases should then be designed using equivalence partitioning, boundary value analysis, decision tables and negative testing.
For example, fund transfer testing should include valid transfers, insufficient balance, invalid account details, maximum transaction limit, duplicate requests, network interruption and transaction timeout. Expected and actual results should be documented.
Defects should be recorded in a repository and analyzed for recurring patterns. Fixed defects should be retested, followed by regression testing of related transaction functions.
Structured testing improves quality by increasing coverage, detecting defects earlier and preventing recurrence. Automation of critical regression tests and continuous review of defect history can further enhance defect detection and banking-system reliability.

Code of Conduct:

I Charuthya P (192421148) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.
Charuthya P
Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand